



SCHARR FILTER USING MATRIX TRANSFORMATION

Group members

1. Thanasoonton Chewakietyingyong
2. Pathomkant Jaithan
3. Nitjaphat Supannagthong
4. Tanawit Oungas
5. Tanakrit Lemontree

Student ID

673040460-5
673040150-0
663040720-4
673040662-3
673040661-5

Digital Media Mathematics (EN841009) Final Project
Faculty of Computer Engineering (Digital Media Engineering)

Preface

Scharr filter using matrix transformation project is as a part of Digital media Mathematics (EN841009) final project on the 12th October 2024. This project was created by the Khon Kaen university student from faculty of Computer Engineering (DME). However, the project focuses on the edge detection method and using matrix transformation for creating the filter with the specific focus on Scharr filter by using Python programming language. Furthermore, this project was created as an innovation to represent the value of computer programming and to offer the concept idea of the basic of computer vision and machine learning. The purpose of this study is to examine the advantages of the Scharr filter and to provide a detailed analysis of formula used during the transformation and demonstrate the program by testing the filter in a complex visual environment by applying the filter into a fantasy themed video.

Throughout the research process, we aim to provide both theories based and practical understanding of the Scharr filter by researching the description of scharr filter, the basic method that use the matrix operation to transform the gray-scale image into the scharr filter. Additionally, we also manually calculate the gradients of the images for further understanding and comparing the results with different analytical methods. This project aims to provide the basic idea of image processing by using computer vision and machine learning.

This project not only created for academic discussion and class presentation on edge detection techniques, but also use for reference source for practicing. However, the project can serve as a resource for those who interested in different edge detection techniques and for those who want to implement for creating the new filter.

Table of Contents

Chapter	Page
Preface	i
Chapter 1 - Introduction	1
1.1 Problem Statement	1
1.2 Objectives of the study	1
Chapter 2 – Literature research	2
2.11 Overview of gradient based edge detection in computer vision	2
2.12 Scharr filter description	2
2.13 Matrix transformation	3
2.14 Matrix transformation in computer vision	5
2.15 Comparison of edge detection filter and applications	5
Chapter 3 - Methodology	7
3.11 Scope and limitations	7
3.12 Brief description of the Fantasy theme video	8
3.13 Structure and algorithm of Scharr filter	8
3.14 Gradient magnitude and edge calculations	10
3.16 Scharr filter using Python computer vision (code)	13
Chapter 4 – Results and Evaluation	17
4.11 Summarization of the results	17
4.12 Visualization and performance of Scharr filter	17
4.13 Problems encountered and attempted solutions	18
Chapter 5 - Conclusion	19
Chapter 6 - References	20

Introduction

1.1 Problem Statement

In today digital world, generating filters is an important process of computer vision. Furthermore, computer vision can include object detection, object classification, image segmentation and feature extraction. Specifically, edge detection is being introduce in order to make filter can detect the boundaries between image areas with different brightness and revealing other characteristics of the image texture. Therefore, our group created a project which transform normal colored video into video that consist of Scharr filter using the process of matrix transformation in machine learning and generating the filter into the video by using the concept of computer vision which include the identification of image, and significant transition in the image intensity.

1.2 Objectives of the study

This project focuses on creating the scharr filter using the concept of matrix transformation in linear algebra; with the application of matrix transformation to the fantasy theme video. This research will cover the mathematical formulas of creating the Scharr filter, by performing matrix calculations that related to gradient magnitude and direction detection, also cover the manually method of computing image gradients for further understanding of the basic image pixel calculation and detection. From this study, we aim to demonstrate the advantages of the Scharr using matrix transformation methods and focus on the potential of the applied applications in computer vision.

Literature research

2.11 Overview of gradient based edge detection filter in computer vision

Filters are tools that enhance the appearance in photos or videos in different ways, for example effecting the light, the contrast, image sharpness, and colors. There are many types of filter method, but in this context, we will be discussing about the gradient edge-based detection filter in computer vision. Gradient-based edge detection used the technique of computer vision for identifying the edge of the image by calculating the magnitude of the gradient. The edge of the image will have a lighter color, for example white, light gray comparing to the others elements inside the image. Furthermore, gradient-based calculation methods focus on calculating the rate of change of intensity (gradient) in an image, identifying regions where the edge located, and the change of the instensities.

The gradient of an image measures the rate at which the intensity changes at a given pixel. In mathematical, the gradient is a vector quantity which include the points in each direction, and has a magnitude that represents the changing rate. Applying this concept to the grayscale images, the gradient at a pixel of an image is being calculate by using the concept of partial derivative of the image intensity correspond to the x and y directions.

2.12 Scharr filter description

Scharr edge detection is a type of filter which associated with the concept of edge detection. However, we use an advanced technique called matrix transformation to find the edge of the image, and computing the image gradient to find the edge of the image in the video. However, the scharr filter needs a scharr operator. Scharr operator is an upgrade version of sobel operator. We can say that scharr filter is an upgrade version of the sobel filter since scharr operator is a form of the Sobel operator that is being upgrade the version and designed to provide more accurate gradient estimation, especially for the edges. Scharr filter have a kernel in both horizontal and vertical direction. However, in the image, there are 3 dimensions, so the kernel of the operator is a 3x3 matrix. Adding these kernel matrices into a grayscale filter will create a scharr filter. In details, when we apply these kernels to the images, the magnitude of the x and y of the kernel is calculated. This indicate the strength of the edge

2.13 Matrix transformation

In mathematics, matrix transformation is a mapping process of vectors from one space to another space using a matrix. In theory, if we have T which is a linear transformation mapping from $IR^n \rightarrow IR^m$ and x is a column vector with n entries. We can say that

$$T(x) = A(x)$$

From the above figure, this is a general formula of finding the transformation of matrix. In the equation, “ A ” represents the $m \times n$ matrix where m is a row and n is a column.

In the matrix transformation, there are many types of solution, however these are the possibilities of finding the solutions from the concept of matrix transformation. Given that $A(x) = b$. We want to find x

- No solution: None of the x value mapped to the b
- One solution / unique solution: One x mapped to b
- Many solutions: Many x mapped to b

After we know the general solution, there are characteristics of a matrix. In order to determine the characteristics, it's important to identify the mapping point of the matrix.

- One to one: In this type of matrix characteristics, there is no point in the domain that are mapped to the same point. These means that the matrix A have pivot position in every column
- On to: Every point of a co-domain has a point that is mapped to it. This means that there can be a possibility of no solution, unique solution, or many solutions which means that every row must have a pivot position in order to create an on to characteristics.

Linear transformation is a type of matrix transformation. Linear transformation between vectors spaces can represent by a matrix. Moreover, linear transformation focuses on two main concepts in order considered the matrix as a linear transformation

- | | | |
|-----|-------------|-------------------------------|
| I) | Additivity | $T(u + v) = T(u) + T(v)$ |
| II) | Homogeneity | $T(c \cdot u) = c \cdot T(u)$ |

If we have many matrices, we can combine each matrix together in a linear transformation and matrix transformation using this theorem

$$A = [T(\mathbf{e}_1) \quad T(\mathbf{e}_2) \quad \cdots \quad T(\mathbf{e}_n)]$$

If we have a linear transformation $T(x)$ in function form, then defining the transformation matrix A is easily done by transforming each vector by T and then inserting the result into the columns of the matrix.

There are many common types of matrix transformation, however these are examples of each common types of matrix transformation in mathematics.

$$\begin{bmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix}$$

I) Matrix translation: Moving points of an object a given distance in a given direction.

$$\begin{bmatrix} x & 0 \\ 0 & y \end{bmatrix}$$

II) Matrix scaling: Increasing and decreasing of the size inside the matrix

$$\text{a) } \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\text{b) } \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

III) Matrix rotation: rotate the object in a given angle, sometimes may associated with trigonometric equations. From the above example, this is an example of matrix rotation.

a) This equation is a counterclockwise rotation

b) This equation is a clockwise rotation

2.14 Matrix transformation in computer vision

$$\begin{bmatrix} xx(1 - \cos \theta) + \cos \theta & yx(1 - \cos \theta) - z \sin \theta & zx(1 - \cos \theta) + y \sin \theta \\ xy(1 - \cos \theta) + z \sin \theta & yy(1 - \cos \theta) + \cos \theta & zy(1 - \cos \theta) - x \sin \theta \\ xz(1 - \cos \theta) - y \sin \theta & yz(1 - \cos \theta) + x \sin \theta & zz(1 - \cos \theta) + \cos \theta \end{bmatrix}$$

The above figure illustrates the matrix is a 3x3 matrix rotation which rotates a vector in 3D space around the axis. However in order to rotate, we define the vector $[x, y, z]$ by an angle θ . Each term in the matrix is responsible for calculating the axis of rotation.

- $[x, y, z]$ are axis where the matrix rotation occurs. For example xx , means the rotation occurs at the xx position. xy means the rotation occurs in xy position.
- $\cos \theta$ controls the rotation of original direction
- $\sin \theta$ controls the perpendicular direction of rotation

Furthermore, this type of matrix rotation can be use for 3D object manipulation in augment reality or in 3D rendering and modeling.

2.15 Comparison of edge detection filter and applications

Making edge detection filter mostly associated with matrix transformation. Comparing the sobel filter and scharr filter. We can see that both filter kernel consists of a gradient magnitude and 3x3 matrix kernel (Since the image / video that transform have x , y , and z dimensions). Scharr filter is a upgrade version of sobel filter. This means that scharr edge detection will be more visible.

The sobel detection filter is considered as one of the most common methods of detecting the edges of the object. This filter use 3x3 matrix kernel to calculate for the magnitude of the gradient of the image pixel. Comparing to the scharr filter, sobel filter has less accuracy than the scharr filter.

Scharr filter is similar to the sobel filter, but it is a more accurate version of the sobel filter. In scharr filter, we change the matrix kernel value in order to make a more accurate. Therefore, this produce a more accurate value especially for edge detection in images with

higher accuracy, using different combinations of different types of masks to improve the rotational symmetry and also reduce the inaccuracy that caused by Sobel operators, the Scharr filter performs well when the accuracy of gradient measurement is required, especially for detecting sharp edges and detailed objects.

Scharr and Sobel filter can be used in different types of applications. However, Sobel filter is used in basic edge detection tasks such as road boundary detection in autonomous driving, face recognition, object detection model. On the other hand, since Scharr filter is considered as a more accurate filter, they are more popular in applications that require high accuracy, such as MRI scan and edge detection in X-ray, shape detection for objects, and being used for advanced robot navigation.

Methodology

3.11 Scope and limitations

Our projects can convert images and video into the filter, but however there are some of the limitations in the projects. This filter can only process image and videos that is only in the format of Mp4 files while the applying the filter to webcam camera and live the filter feature isn't available yet. Although the filter can be applied to the video and images, but however, it does not explore the hardware features and does not cover much of the machine learning edge detection method.

3.12 Brief description of the Fantasy theme video

Our video is about a battle between 2 sides, each side have 2 people in the team. We used our own unique enchanted weapons to fight. One the teammates in each team are defeated, so the remaining people have to duel. After we got a winner, a secret man hit the winner a brick... making him the real winner.

3.13 Structure and algorithm of scharr filter

In order to create a scharr, at first, we need to apply the grayscale to the video. The grayscale is being represent in the RGB values.

$$Gray = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

From the equation, R, G and B represents the channels of red, green and blue. After we apply the grayscale algorithm to the frame, now, we can identify that the image color in the video is now gray.

After converting the image to grayscale, we can apply the Scharr filter by transforming the image with two specific kernel matrices one for the x-direction and one for the y-direction. These kernels are being designed to detect changes in pixel intensity along their axis, which helps in identifying horizontal and vertical edges within the image.

For illustration, the formula represents the Sobel operator used for Sobel edge detection, both the Scharr and Sobel filters considered as the purpose of edge detection in image processing. Although they are closely related and both of them are edges detection filter based on intensity gradients and gradient magnitude. They use different kernel and matrices which can result in different performance and sensitivity. On the other hand, they share similar objectives, Scharr and Sobel are techniques within the scope of edge detection.

Sobel operator

$$g(x) = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$
$$g(y) = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

The above formula shows the matrix for sobel operator. This matrix is being use for changing the grayscale image and videos into sobel filter. However, there are 2 kernels with 3x3 matrix since the image is 3 dimensional. Starting with the x-direction kernel $g(x)$ detects vertical edges (intensity changes from left to right) and $g(y)$ detects the horizontal edges (intensity changes from top to bottom).

For summarization:

- $g(x)$, x-direction: Use for detecting the vertical edges which identify the intensity changing from left to right. If there is a significant difference in horizontal pixel intensity, the x-direction kernel will produce a strong response.
- $g(y)$, y-direction: Use for detecting the horizontal edges. This kernel can identify the changes in the intensity that occur from top to bottom across pixels. If there is a significant difference in vertical pixel intensity, the x-direction kernel will produce a clear response

Applying the concept of sobel operation, we can create the scharr filter. Scharr filter is an upgrade version of the sobel filter. However, the detection method is more sensitive comparing to the sobel detection method.

Scharr operator

$$g(x) = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

$$g(y) = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

From the scharr operator, we can identify that both operators are somewhat similar. Both scharr and sobel use the concept of matrix transformation, moreover, when changing the direction of the kernel from $g(x)$ to $g(y)$, we transpose the matrix to get another dimension. In scharr filter, we convert the number 1 (from sobel operator) into 3 and 2 (from sobel operator) into 10 for more accurate edge detection method.

After we apply these kernels (for both scharr and sobel operator) in both the x and y directions in grayscale image or videos, we can see the detailed of the edges present in both directions and then combine these edges from the x and y directions to calculate the overall gradient magnitude, which provides a edge map of the image. This process will enables accurate edge detection, which is important in many computer vision applications, such as object recognition, image segmentation, and feature extraction.

3.14 Gradient magnitude and edge calculations

After applying the kernels into the grayscale, the program will calculate for the gradient magnitude by using the below formula

$$G = \sqrt{g(x)^2 + g(y)^2}$$

From the above formula shown, “G” illustrates gradient magnitude, $g(x)$ and $g(y)$ are vertical edge and horizon edge kernel (x and y direction). This formula will check for the edge and the magnitude of the gradient.

$$\theta = \arctan\left(\frac{g(x)}{g(y)}\right)$$

We can also find the directional angle of the gradient orientation and magnitude by using the above formula. For example, let $g(x) = 0$ and $g(y) = -255$, using the above formula, we can produce a gradient magnitude at the position 255 in the pixel matrix in the image, and also produce the gradient orientation at -90 degrees.

We know how to find the gradient magnitude and direction, but how do we apply to the matrix? The below statement will demonstrate the calculation for detecting the strength of the edge and determine whether the intensity increase or decreases using manual computation method. The below method will explain how to calculate the gradient intensity

Suppose that we have a pixel (in matrix format), and we want to calculate the intensity value of the intensity whether it is increasing or decreasing.

$$\begin{bmatrix} 50 & 50 & 100 & 100 \\ 50 & 50 & 100 & 100 \\ 50 & 50 & 100 & 100 \\ 50 & 50 & 100 & 100 \end{bmatrix}$$

These are the pixels that we selected for calculating the intensity, suppose we want to calculate the following area (3x3) matrix inside the pixel for both $g(x)$ and $g(y)$

$$\begin{bmatrix} 50 & 50 & 100 \\ 50 & 50 & 100 \\ 50 & 50 & 100 \end{bmatrix} \text{ and } \begin{bmatrix} 100 & 100 & 50 \\ 100 & 100 & 50 \\ 100 & 100 & 50 \end{bmatrix}$$

After we selected the pixel area, we multiply the scharr operator to the pixel for both g(x) and g(y) axis

For g(y) axis:

$$\begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix} \cdot \begin{bmatrix} 50 & 50 & 100 \\ 50 & 50 & 100 \\ 50 & 50 & 100 \end{bmatrix}$$

$$(-3)(50) + (-10)(50) + (-3)(100) + (0)(50) + (0)(50) + (0)(50) + 3(50) + (3)(50) + (3)(100)$$

$$= -150 - 500 + 300 + 150 + 500 + 300$$

$$= 0$$

For g(x)

$$\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix} \cdot \begin{bmatrix} 100 & 100 & 50 \\ 100 & 100 & 50 \\ 100 & 100 & 50 \end{bmatrix}$$

$$-300 + 0 + 150 - 1000 + 0 + 500 - 300 + 0 + 150$$

$$= -200$$

After the calculation in both g(y) and g(x) axis, the results show that in the g(y) axis, there is no intensity value change, while in the g(x) axis, the intensity decrease because the value is negative.

Taking another example, for calculating the intensity

$$\begin{bmatrix} 14 & 51 & 46 & 201 & 172 & 183 \\ 21 & 67 & 3 & 194 & 164 & 191 \\ 12 & 35 & 38 & 196 & 175 & 229 \\ 39 & 25 & 23 & 152 & 156 & 216 \\ 39 & 10 & 62 & 167 & 144 & 167 \\ 69 & 35 & 20 & 162 & 186 & 219 \end{bmatrix}$$

However, we selected some region out from the image pixel and calculate with the scharr operator

$$\begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix} \cdot \text{region selected}, \quad \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix} \cdot \text{region selected}$$

After the calculation process, the intensity value for $g(x) = 44$ and $g(y) = 576$. This means the intensity in both dimensions are increasing. Furthermore, we can find the magnitude of the gradient

$$\sqrt{44^2 + 576^2} = 577$$

From calculating the magnitude of the gradient, we can identify the edge location. In addition, edge location will locate at the edge strength of 577 pixels.

Functions of operation in $g(x)$ and $g(y)$ kernel

$$g(x) = \begin{bmatrix} -3 & 0 & 3 \\ -10 & 0 & 10 \\ -3 & 0 & 3 \end{bmatrix}$$

- The first column detects the strong decrease in the gradient intensity
- The second column doesn't contribute to the detection, stay the same
- The third column increase the gradient intensity

However, in the $g(y)$ kernel, this is the opposite of $g(x)$ kernel (transpose $g(x)$ kernel)

$$g(y) = \begin{bmatrix} -3 & -10 & -3 \\ 0 & 0 & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

- The first row detects the strong decrease in gradient intensity
- The second row doesn't contribute with the detection
- The third row increase the gradient intensity

Furthermore, if the matrix (the image pixel) is a zero matrix. This means that there is no edge, on the other hand, if there is a value rather than zero in the matrix there will be an edge calculation.

3.16 Scharr filter using Python computer vision (code)

Implementing the scharr filter concept into computer vision requires Python programming. The below figures explain the concept of programming applying to the scharr filter calculation.

```
import imageio
import numpy as np
```

On the first step, we import different library such as imageio and numpy for scharr operator calculation and processing images

- Numpy = Working with arrays, especially for the matrix calculation for this project.
- Imageio = Use for image processing and video processing. Converting the images, including the animated images such as videos

```
resolution_scale = 1
fps_scale = 1
```

After importing libraries. Set the resolution scale and FPS scale. In the above code, setting both resolution and FPS variable as 1 will make the quality of video and the framerate of the video after processing same as the original video. However, we cannot set the value higher than 1, but if we set the value below 1 will decrease the quality of the video after processing

```
def rgb_to_gray(frame):
    return np.dot(frame[..., :3], [0.2989, 0.5870, 0.1140]).astype(np.uint8)
```

Create a function for changing the video color to grayscale with 1 parameter. In the function, the program will perform the calculation similar to below example

$$Gray = 0.2989 \cdot Red + 0.5870 \cdot Green + 0.1140 \cdot Blue$$

- np.dot performs a dot product calculation that we learn in class to calculate the grayscale values
- [..., :3]: the “...” means the original value of the array. “:3” refers to dimensions
- np.astype(uint8) = change the datatype of the array element to the specific type . The result is cast to uint8 which returns a 8 bit integer from 0-255

```

def scharr(image):
    vertical = np.array([
        [-3, 0, 3],
        [-10, 0, 10],
        [-3, 0, 3]
    ])

    horizontal = np.array([
        [-3, -10, -3],
        [0, 0, 0],
        [3, 10, 3]
    ])

    height, width = image.shape
    output_image = np.zeros((height, width))

    for i in range(1, height - 1):
        for j in range(1, width - 1):
            region = image[i-1:i+2, j-1:j+2]
            grad_x = np.sum(region * vertical)
            grad_y = np.sum(region * horizontal)
            magnitude = np.sqrt(grad_x**2 + grad_y**2)
            output_image[i, j] = np.clip(magnitude, 0, 255)

    return output_image.astype(np.uint8)

```

After making the grayscale conversion function, create a scharr function with 1 specific parameter (the argument should come from the video that already transform to grayscale). Create a horizontal and vertical edges calculation using numpy arrays. This function will iterate through each pixel (excluding the edges), uses a kernel to compute the gradient, and computes the magnitude of the gradient, which represents the strength of the edge.

- Height, width: in this process, we get the height and the width of the image from the image parameter
- Using loops to iterate
- np.clip: use for limiting the values of the arrays. The external value will be transferred to the internal array values

`output_image` are already in the form of arrays. Converting array into np.uint8

```
input_video_path = 'videos/filter_fantasy.mp4'

output_video_path = 'Processed_video/filter_fantasy_processed.mp4'

reader = imageio.get_reader(input_video_path)
fps = reader.get_meta_data()['fps']
frame_size = reader.get_meta_data()['size']

target_fps = int(fps * fps_scale)
target_resolution = (int(frame_size[0] * resolution_scale),
int(frame_size[1] * resolution_scale))

frame_skip = int(fps / target_fps)

writer = imageio.get_writer(output_video_path, fps=target_fps)
```

Specify the video input and output video. For the input video, we insert the video path that is being uploaded **before** filtering. For the output path, create a new folder for the output, and specify the name for the output video (the video name after processing).

Read the input video, the frame size and get the FPS of the video using dictionary method (accessing dictionary). However, apply the resolution to the video for filtering method (this can be change via `resolution_scale` and `fps_scale`). Then we divide the `fps/ target_fps` and create a write object for saving processed video with desire framerate.

```
try:
    frame_count = 0
    for frame in reader:
        if frame_count % frame_skip == 0:
            print(f"Processing Frame #{frame_count + 1}")

            resized_frame = frame[:, :int(1/resolution_scale), :int(1/resolution_scale), :]

            gray_frame = rgb_to_gray(resized_frame)

            scharr_frame = scharr(gray_frame)
            writer.append_data(scharr_frame)

            frame_count += 1
finally:
    writer.close()
```

The code iterates through each frame of the video. Furthermore, it processes every `frame_skip` frame based on the desired FPS. We also use `try`, `catch`, and `finally` method for error handling (`finally` is being use for leaving `try` statement). Initialize the `frame_count` as 0 and increment through the loop while the frame is being processed. Resize the frame using list comprehensions throughout the looping statement (this will reduce the frame size). After that, we apply the grayscale to the original video following with the `scharr` filter into the grayscale video. Increment the framecount (`frame_count = frame_count + 1`) throughout the process until the video processing ends. Finally, close the writer object using `writer.close`, so that the video processing ends.

Results and Evaluation

4.11 Summarization of the results

As a result, we have successfully applied the scharr filter (edge detection) using matrix transformation method applying to the video with the implementation by python programming language. Furthermore, we use the weight and sum approach to convert the video to grayscale which can reduce the complexity of filtering, focusing on intensity variations, and also reduce the colors into gray, white and black color, so that the filter can be apply easily.

To make the processing time faster, we can apply the resolution and framerate scale. However, in our project, we use the full resolution and fps scale (100%, no reduction). The overall frame skip ratio was computed to match the original frames per second (FPS). The scharr edge detection, also works, making clearer and smoother edges comparing to the sobel edge detection (mostly for diagonal edges and sharp edges).

4.12 Visualization and performance of Scharr filter



From the above image, the scharr filter works well for video and image processing. The arrays is being applied to the video, and processed out to the output file. Since the video is not 100% clearly taken, there would be a blur effects which can leads to small white edges all around the images.

4.13 Problems encountered and attempted solutions

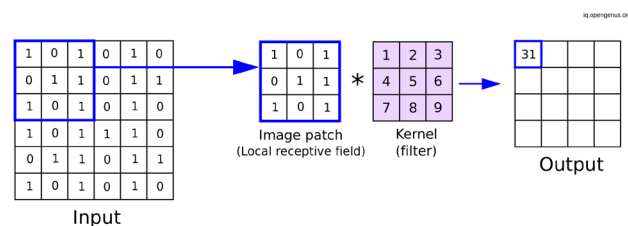
During the process of implementation of scharr filter, there are many problems encountered. At first, the video processing took very long time due to the complexity of applying the scharr filter to each pixel of the images. There are 2 ways of solving the problem. The first solution is that, reduce the FPS scale and resolution scale. This method can speed up the processing time, and reduce the critical details of the video. The second method that we use is that, we import the library called **scipy**, calling the library by the method of

```
from scipy.signal import convolve2d
```

This method can help speed up the processing time by convolving the 2-dimensional arrays. From the main code, we implement as the below.

```
def scharr(image):  
    vertical = np.array([  
        [-3, 0, 3],  
        [-10, 0, 10],  
        [-3, 0, 3]  
    ], dtype=np.float32)  
    horizontal = np.array([  
        [-3, -10, -3],  
        [0, 0, 0],  
        [3, 10, 3]  
    ], dtype=np.float32)  
  
    grad_x = convolve2d(image, vertical, mode="same", boundary="wrap")  
    grad_y = convolve2d(image, horizontal, mode="same", boundary="wrap")  
    magnitude = np.sqrt(grad_x**2 + grad_y**2)  
    return np.clip(magnitude, 0, 255).astype(np.uint8)
```

(This method is just for fast processing to test whether the filter is correctly applied to the image. We use this only for the filter testing, but for the final product, we use the normal method shown in the methodology part of the research)



Conclusion

In conclusion, generating filter is considered as one of the main ideas of computer vision. Furthermore, computer vision can be used for object detection, object classification, image segmentation and feature extraction. From the project, we have successfully created a scharr filter for edge detection using matrix transformation method applying with the python programming method.

As a result, our scharr filter project using matrix transformation can guide the way for further implementation and improvement of method of edge detection techniques, by including the use of other additional filters and different image processing methods. Overall, the methods that we use and created throughout this project can be use for a future research and other applications for computer vision and image processing

Reference

Edge Detection in Image Processing: An Introduction. (2024, June 14). Roboflow Blog.

<https://blog.roboflow.com/edge-detection/>

Rosebrock, A. (2021, May 12). Image Gradients with OpenCV (Sobel and Scharr). *PyImageSearch*.

<https://pyimagesearch.com/2021/05/12/image-gradients-with-opencv-sobel-and-scharr/>

Transformation Matrix. (2024, May 20). GeeksforGeeks.

<https://www.geeksforgeeks.org/transformation-matrix/>

Python Arrays. (2018, December 26). GeeksforGeeks. <https://www.geeksforgeeks.org/python-arrays/>

Getting Started With ImageIO Library in Python. (2020, August 14). GeeksforGeeks.

<https://www.geeksforgeeks.org/getting-started-with-imageio-library-in-python/>